

CrowdImprint Supplementary Information

Marilena Lemonari¹ · Panayiotis Charalambous² · Julien Pettr ³ · Yiorgos Chrysanthou¹

This document provides supplementary information, details, and additional examples of CrowdImprint’s methodology and results. Specifically, the additional material regards the structure of the input, the training of the model, and inference and case study secondary results and implementation details.

A. Input Structure

A.1. Input Feature Engineering Study

Figure 1 shows examples of the 3 different image-based input structures studied during the input feature engineering experiments.

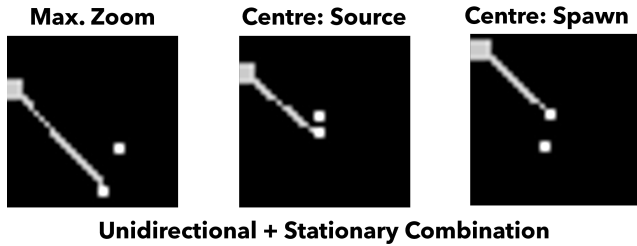


Fig. 1 Feature Engineering for Inputs.

M. Lemonari, P. Charalambous, Y. Chrysanthou
CYENS - Centre Of Excellence, Nicosia, Cyprus
E-mail: m.lemonari@cyens.org.cy, p.charalambous@cyens.org.cy, y.chrysanthou@cyens.org.cy

M. Lemonari
University of Cyprus, Nicosia, Cyprus
E-mail: lemonari.marilena@ucy.ac.cy

J. Pettr 
Inria, University of Rennes, CNRS, IRISA, Rennes, France
E-mail: julien.pettr @inria.fr

A.2. Input Information Ablation Study

Figure 2 shows example images of the input information ablation study. Specifically, the importance of 3 types of information are examined: presence of **source** position, agent **spawn** position, and **speed** information - colouring more pixels when agent is static.

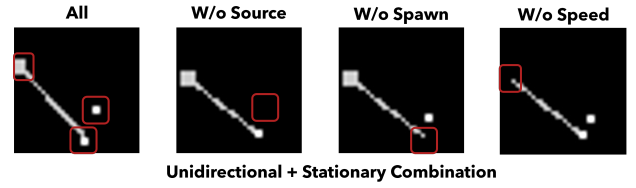


Fig. 2 Input Information Ablation.

A.3. Data Representation

Figure 3 shows additional examples of trajectory images corresponding to different selections of core behaviour combinations.

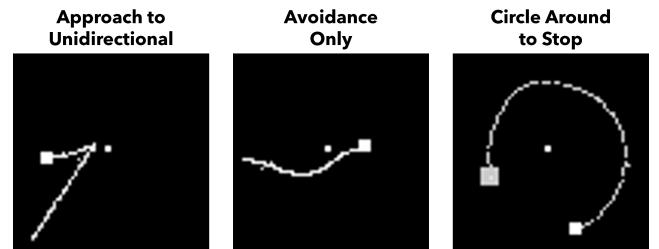


Fig. 3 Training Image Examples.

B. Model Training

B.1. Model Architecture

Name	Layers	Input	Output
Classification CNN	2D Convolution	1	16
	2D Convolution	16	32
	2D Convolution	32	64
	Linear	4096	1024
	Linear	1024	512
	Linear	512	128
Siamese Network Encoder	2D Convolution	1	64
	2D Convolution	64	128
	Linear	128	128

Table 1 Model Architecture.

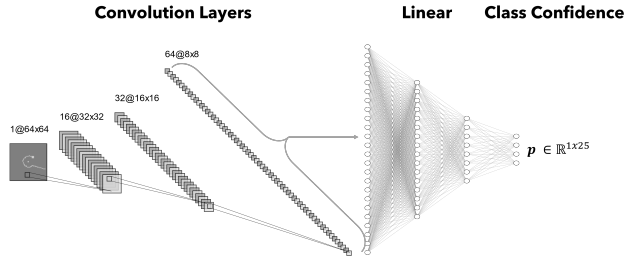


Fig. 4 Classification CNN Architecture.

Table 1 and Figure 4 show the CNN architecture. All convolution layers have 2D batch normalization, followed by max pooling and dropout (0.4). All fully connected layers had 1D batch normalization followed by 0.6 dropout rate. The output of the last layers passed through a $\log_{softmax}$ operation.

B.2. Confusion Matrices

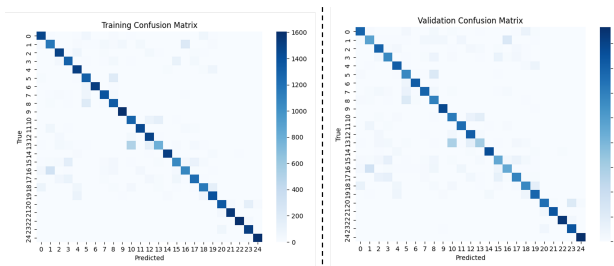


Fig. 5 Training and validation confusion matrices.

As indicated in Figure 5, the model mostly mispredicts the “Circle Around-Avoid” as “Circle Around-Continue” with

biggest confusions generally coming from continuing with avoidance (and vice versa). This makes sense considering the case where an individual has to avoid an object which, however, is not in close proximity with.

B.3. Training Pipeline

Figure 6 shows the training pipeline. Firstly two of the core behaviours are sampled (could be the same core twice). Then the scenario is set-up by randomly choosing (among others) time of field switch, simulation duration, and agent spawn. The sampled scenario is then inputted into the compatible, underlying simulator (InF-UMANS) to produce the corresponding trajectories, which we use to build our encoded image representation. The inputs to our model are then these encoded trajectory images, which are passed through our classification CNN to output confidence probabilities for each of the possible core behaviour combinations. We train our model to maximize the probability of the ground truth class for each sample.

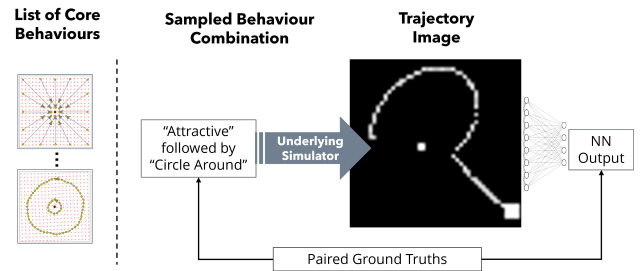


Fig. 6 Training Pipeline.

C. Applications

C.1. Inference on Large Dataset

We perform inference on two larger datasets (Figure 7). The more data used to implicitly define the intended crowd behavior, the more robust the resulting distribution. We note, that since our model is source-aware, it is inherently highly sensitive to the source positioning. So, considering that large existing crowd trajectory datasets are not source-aware, we only use these two examples to showcase the model’s potential.

C.2. Model Adaptability

In Figure 8, we show an additional example of iterating our model two times over a pair of segmented motions. This

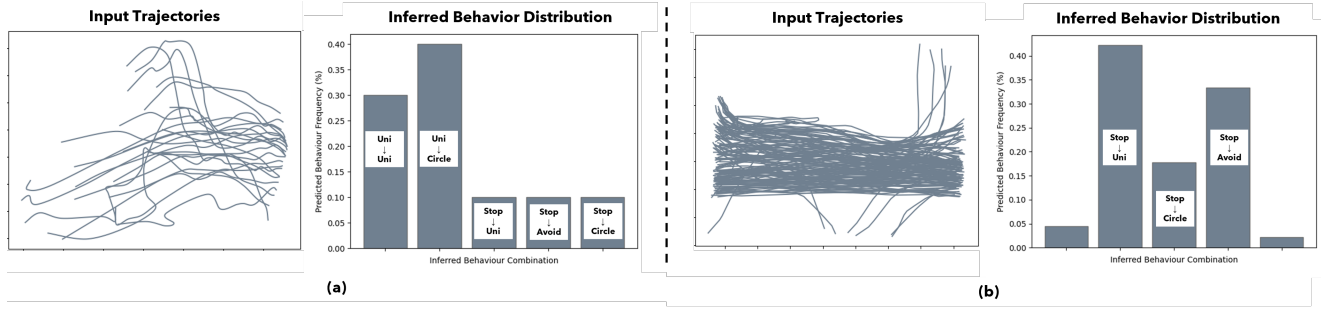


Fig. 7 Inference with Larger Source Data: (a) from the Flock scenario (Church Yard), (b) from the Zara scenario (Commercial Street).

example yields a combined behaviour of continuing to approaching to finally stopping. This stopping effects was not captured during the single model iteration which makes sense since it is a less prominent effect.

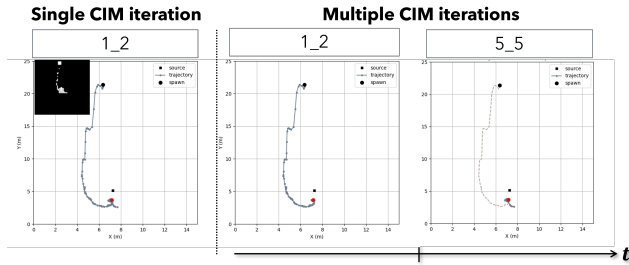


Fig. 8 Model iterations: longer motions can be split into smaller segments and passed through CIM, outputting additional behaviours.

C.3. Metrics on Specific Scenarios

We calculate the dedicated metrics for stop, continue, circling, avoid, and approach for the data of the specific scenarios that were collected: exhibit, food court, ticket machine/atm, guard, and friend. We do not have the ground truth behaviours, and so we visualise the metrics per-scenario using radar plots in Figure 10. We can see that the generated data fairly follow the patterns of the collected, especially in food court, atm, friends.

C.4. CCP and OG InFs Simulations

To obtain the trajectories of a predefined scenario, e.g., museum exhibit, from the OG InF framework, we firstly define a custom (arbitrarily intricate) InF that is specific to the intended effect we want. Then we specify scenario parameter via a set of xml files, and generate the corresponding trajectories. Similarly, for the CCP, we interact with its UI to dynamically define desirable profiles that match the behaviours we wish, and export the trajectories. All trajectories are then

encoded (in images) and passed through our trained model (Figure 9). Table 2 presents results for the “inspector” sce-

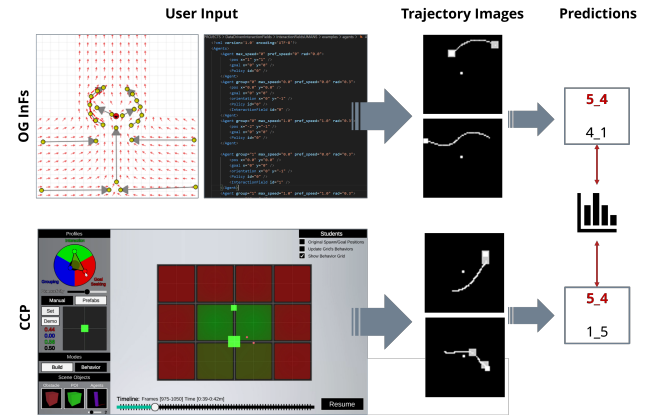


Fig. 9 Process outline of obtaining predictions from CCP and OG InFs.

nario. We observe that CCP generates trajectories with relatively frequent stopping, which is logical considering the need to dynamically adjust the UI sliders to alter behaviour. For OG InFs, the user has to sketch either a single InF to affect all agents, or targeted fields for groups of agents, and then initialise them according to their scripted `xmls`. Ultimately, this makes it challenging to generate agents that all behave as intended, e.g., avoiding the inspector before interacting with them and then dispersing (4_1).

Source		Predictions
OG InFs	<i>ag1</i>	5_4* ¹
	<i>ag2</i>	4_1
CCP	<i>ag1</i>	5_4*
	<i>ag2</i>	1_5
SL	<i>ag1</i>	5_4*
	<i>ag2</i>	1_2*

*exists in the inferred behaviour distribution.

Table 2 Prediction Similarity between Simulators (“Inspector” Scenario).

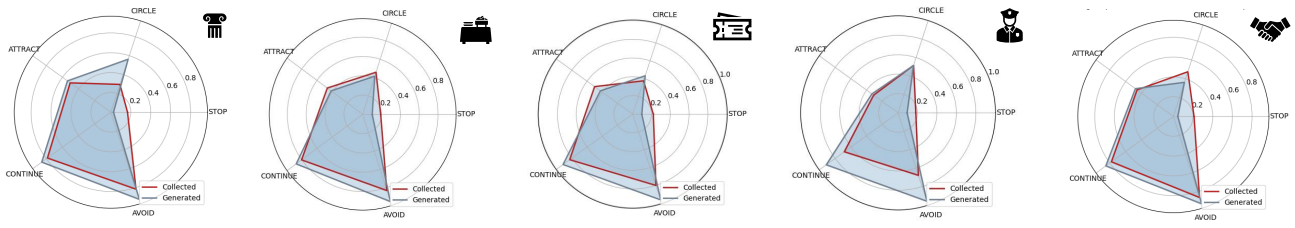


Fig. 10 Trajectory metrics for unseen scenarios summarised in radar plot, per-scenario.

C.5. Siamese Model

Table 1 (bottom) together with Figure 11, outline the siamese model design.

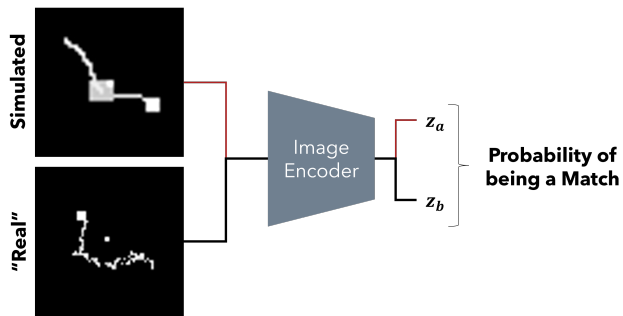


Fig. 11 Siamese model architecture.

¹ {1: Continue, 2: Approach, 3: Circling, 4: Avoid, 5: Stop}.